МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1**
**по курсу “Компьютерная графика”**

***Основы 3D графики***

Выполнил: М. Н. Осипов
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

## Условие

Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортографической проекцией и аффинными преобразованиями.

Программа должна работать в реальном времени, с возможностью динамической смены проекции и трансформаций объектов. Все объекты должны корректно отрисовываться с учетом проекции и иметь возможность взаимодействия с пользователем.

Вариант 5 (Сфера (~100 вершин)).

Дополнительные задания:

1. Обеспечьте переключение матрицы проекции (ортографическая и перспективная) в интерфейсе;
2. Добавить элементы интерфейса для изменения позиции, поворота и растяжения фигуры;
3. Добавить возможность движения и поворота фигуры по сложной траектории, сделать элемент интерфейса для паузы и воспроизведения “анимации”, а также возможность изменять скорость “анимации” и другие параметры траектории (например радиус траектории и др.);
4. Добавить возможность изменять цвет фигуры с помощью элементов интерфейса (в ImGUI есть ColorEdit для этого);
5. Назначить процедурно цвета для каждой вершины, например в зависимости от позиции вершин в локальной системе координат фигуры, тогда возможность изменять цвет с помощью элементов интерфейса будет умножать выбранный цвет на цвет вершин.

## Метод решения

Сфера строится через параметризацию в сферических координатах: полярный угол от [0, pi] и азимутальный [0,2pi] разбиваются на 9*9 ячеек, что даёт (9+1)*(9+1) = 100 вершин и 486 индексов (по два треугольника на ячейку). Каждая вершина получает процедурный цветn*0.5+0.5 от своей нормали. Положение вершин на экране вычисляется шейдером как P*V*M*vec4(position,

1.0), где M — композиция переноса, поворотов и масштаба, V — матрица камеры (glm::lookAt), P — проекция, перспективная (glm::perspectiveRH_ZO) или ортографическая (glm::orthoRH_ZO); вариант RH_ZO выбран из-за Vulkan-диапазона глубины [0,1]. Вершины и индексы загружаются в device-local память через staging buffer: данные копируются в host-visible буфер, затем командой vkCmdCopyBuffer переносятся в основной буфер. Матрицы каждый кадр пишутся в замапленный uniform buffer через memcpy и становятся доступны вершинному шейдеру через VkDescriptorSet (binding 0). Цвет из UI передаётся во фрагментный шейдер через push constants и умножается там на процедурный цвет вершины. Отрисовка выполняется командой vkCmdDrawIndexed внутри render pass'a с включённым depth-тестом и back-face culling; смена проекции, трансформации, цвет и анимация по спиральной траектории управляются через ImGui-слайдеры, радио-кнопки и ColorEdit4.

## Результаты

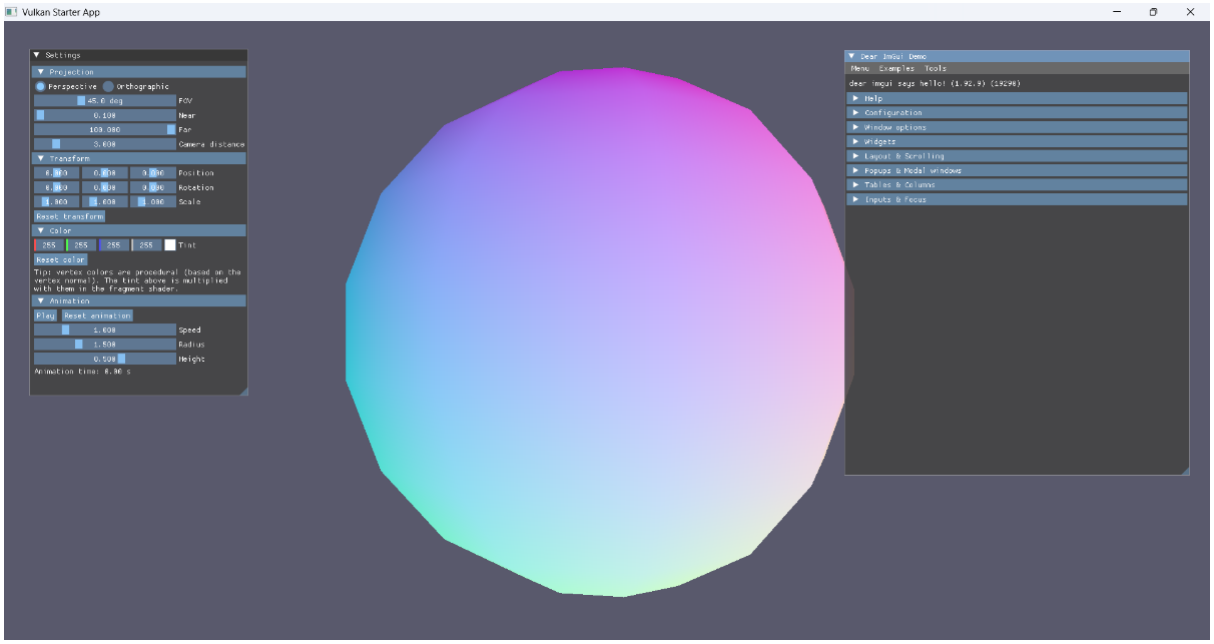



**Рисунок 1.** Общий вид приложения: сфера из 100 вершин с процедурной раскраской по нормалям, окно Settings с настройками проекции, трансформаций, цвета и анимации, а также демо-окно ImGui.

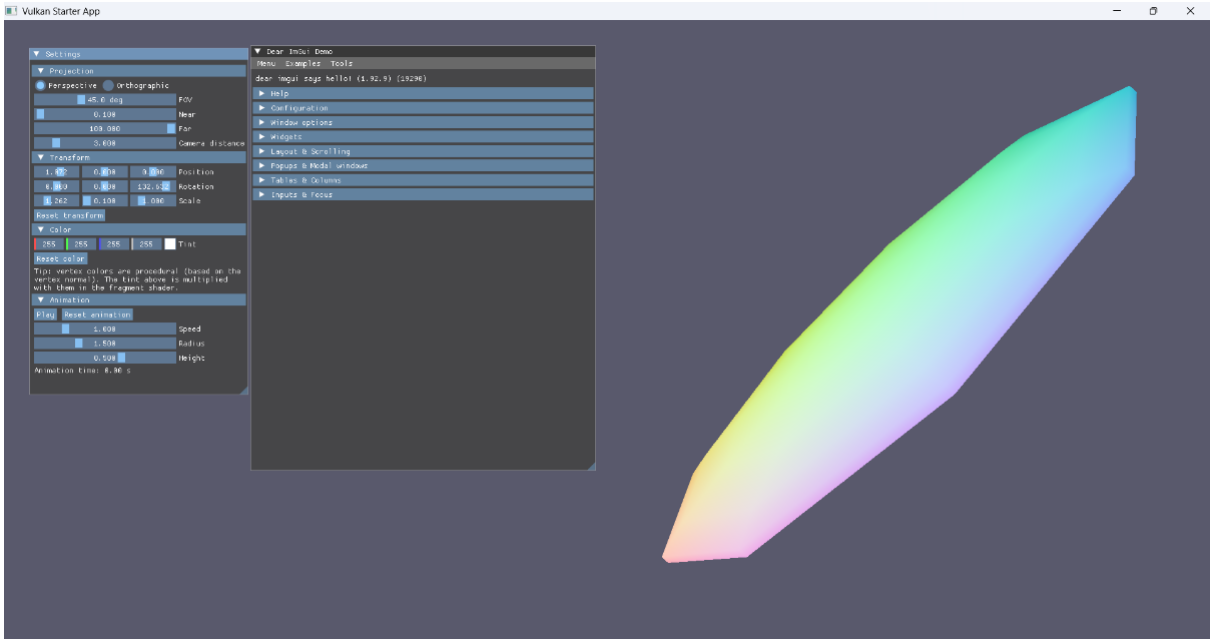



**Рисунок 2.** Применение аффинных преобразований: сфера растянута слайдером Scale, повёрнута и перенесена слайдерами Position и Rotation — видно, как фигура превратилась в вытянутый эллипсоид и сместилась в правую часть экрана.

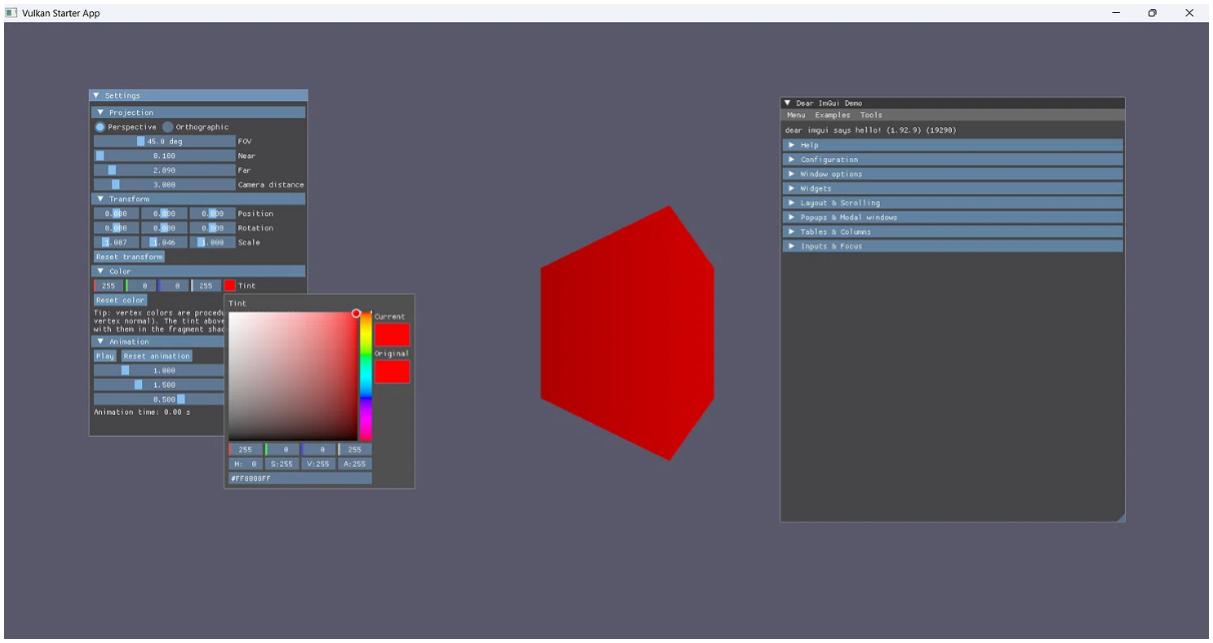



**Рисунок 3.** Изменение цвета фигуры через ColorEdit4: в окне палитры выбран красный цвет (RGB 255, 0, 0), который передаётся во фрагментный шейдер через push constants и умножается на процедурный цвет вершин — сфера окрасилась в красный оттенок. Дополнительно слайдер Far выставлен в 2.0, что демонстрирует отсечение геометрии дальней плоскостью проекции.

## Выводы

Я познакомился с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научился работать с матрицами перспективы, ортографической проекцией и аффинными преобразованиями.